

PROJET DE CONFIGURATION

Unitree 4D LiDAR L1 sous ROS 2 Humble

Cadrage technique, environnement, méthode de travail et journal initial de la conversation

Élément	Valeur
Auteur / utilisateur	Pascual
Contexte	Stage de robotique - ISR / HcMR Lab, Universidade de Coimbra
Machine cible	isr-SATELLITE-P50-C
Dossier de projet	~/unitree_l1_project
Système hôte constaté	Ubuntu 24.04 (Noble), architecture 64 bits
Date de cette version	15 juillet 2026
État global	Codex installé et authentifié; Remote Control connecté; association du client à finaliser; ROS et LiDAR non encore configurés dans ce dossier

But final. Faire fonctionner le Unitree 4D LiDAR L1, lire et contrôler ses données 4D sous ROS 2 Humble, enregistrer les mesures, puis construire et exporter une carte 3D exploitable d'une pièce.

Ce document est une exportation structurée de la conversation. Il ne remplace pas encore le compte rendu final d'installation: il fixe le périmètre, les décisions, les vérifications déjà effectuées et la feuille de route. Les codes d'association et les secrets ont volontairement été supprimés.

Version impression: noir, blanc et gris; format A4; marges et tableaux conçus pour une impression recto verso.

Sommaire

Sommaire	2
1. Synthèse du projet	3
1.1 Résultats attendus	3
1.2 Hors périmètre immédiat	3
2. Environnement constaté et contraintes	3
2.1 Décision de compatibilité principale	4
2.2 Architecture cible	4
3. Structure recommandée du projet	4
4. Chemin de réalisation détaillé	4
5. Contrôles ROS 2 à effectuer	7
6. Problèmes prévisibles et stratégie de résolution	8
7. Conditions générales de travail	8
8. Journal initial des commandes et incidents	9
8.1 Diagnostic technique conservé	9
9. Export structuré de la conversation	9
10. Critères de validation et listes de contrôle	10
10.1 Avant de brancher ou configurer le LIDAR	10
10.2 Validation du pilote	10
10.3 Validation de la cartographie	10
10.4 Livrables finaux	10
11. Registre initial des décisions	11
12. Références techniques de départ	11

1. Synthèse du projet

Le projet est volontairement limité au capteur. Le Jackal et l'intégration mobile complète sont hors périmètre pour cette première étape. Le système attendu doit recevoir les mesures du LiDAR, publier les données ROS 2, vérifier leur qualité, enregistrer des jeux de données et produire une carte 3D d'une pièce.

La configuration doit rester pédagogique et reproductible. Chaque commande Linux ou ROS sera expliquée avant son exécution: rôle, raison du choix, effet attendu et précautions. Toutes les commandes, erreurs et corrections devront être inscrites dans un journal de configuration, qui servira ensuite à produire le rapport final.

1.1 Résultats attendus

- Un environnement ROS 2 Humble reproductible et compatible avec la machine Ubuntu 24.04.
- Le pilote Unitree compilé et lancé avec une configuration explicitement documentée.
- Un nuage de points ROS 2 de type sensor_msgs/msg/PointCloud2, attendu sur /unilidar/cloud.
- Des mesures inertielles de type sensor_msgs/msg/Imu, attendues sur /unilidar/imu.
- Une visualisation stable dans RViz2 et des contrôles de fréquence, horodatage, champs et repères.
- Un petit nœud ROS 2 de contrôle, développé dans le projet, pour résumer l'état des données.
- Un ou plusieurs enregistrements rosbag2 reproductibles.
- Une trajectoire LiDAR-inertielle et une carte 3D exportée, par exemple en PCD.
- Un rapport PDF final décrivant installation, commandes, incidents, solutions et justification des choix.

1.2 Hors périmètre immédiat

- Commande, navigation ou intégration du robot Clearpath Jackal.
- Déploiement permanent sur une architecture embarquée de production.
- Fusion avec d'autres capteurs tant que le LiDAR et son IMU ne sont pas validés seuls.

2. Environnement constaté et contraintes

Composant	État constaté	Conséquence
Machine	isr-SATELLITE-P50-C	Hôte de travail du projet et futur point de connexion au LiDAR.
Système	Ubuntu 24.04 / Noble, 64 bits	ROS 2 Humble n'est pas la distribution binaire native de Noble.
Dossier	~/unitree_l1_project	Toutes les sources, journaux, cartes et scripts doivent rester organisés ici.
Codex CLI	0.144.4, installation autonome cohérente	Agent local disponible pour inspecter, modifier, compiler et documenter.
Authentification	Connexion ChatGPT réussie	Les erreurs HTTP 401 dues à l'absence d'identifiants sont résolues.
Remote Control	Serveur connecté; association manuelle encore à stabiliser	La configuration locale peut continuer même si Remote reste indisponible.
ROS 2	Non installé dans le nouveau cadre de travail au début de la conversation	L'installation doit être contrôlée et documentée avant le pilote.
LiDAR	Modèle Unitree 4D LiDAR L1; branchement et accès non encore validés	Ne pas supposer le nom de périphérique ou l'interface avant inspection.

2.1 Décision de compatibilité principale

ROS 2 Humble cible officiellement Ubuntu 22.04 (Jammy), tandis qu'Ubuntu 24.04 (Noble) est la plateforme officielle de ROS 2 Jazzy. Comme le projet exige Humble et que les dépôts Unitree / Point-LIO retenus ont été testés dans cet environnement, l'option de référence est: Ubuntu 24.04 comme hôte, avec un conteneur Docker Ubuntu 22.04 + ROS 2 Humble.

Cette décision reste à valider après l'inventaire du PC. Elle limite les mélanges de dépendances sur l'hôte, facilite la reproduction et permet de conserver l'objectif Humble. Le conteneur devra toutefois recevoir explicitement l'accès au périphérique LiDAR, au réseau si nécessaire et à l'affichage RViz2.

2.2 Architecture cible

Capteur Unitree L1 LiDAR + IMU	Interface à confirmer Détection et permissions hôte	Docker Ubuntu 22.04 ROS 2 Humble	Pilote Unitree cloud + imu
Contrôles topics, QoS, fréquence, timestamps, frames	Enregistrement rosvbag2	Odometry Point-LIO ROS 2	Sortie trajectoire + carte PCD

Lecture logique: le capteur alimente le pilote; les données sont contrôlées et enregistrées; Point-LIO estime le mouvement et accumule les points dans un repère de carte.

3. Structure recommandée du projet

```
unitree_ll_project/  
|-- README.md # objectif, démarrage rapide, état  
|-- AGENTS.md # règles durables pour Codex  
|-- docker/ # image Ubuntu 22.04 + Humble  
| |-- Dockerfile  
| |-- compose.yaml  
|-- ros2_ws/ # workspace colcon  
| |-- src/  
| |-- unilidar_sdk/ # pilote Unitree  
| |-- ll_monitor/ # nœud de contrôle développé  
| |-- point_lio_ros2/ # odométrie / mapping  
|-- config/ # paramètres et calibrations  
|-- launch/ # lancements reproductibles  
|-- scripts/ # vérifications et export  
|-- bags/ # enregistrements rosvbag2  
|-- maps/ # PCD et résultats  
|-- logs/ # sorties techniques utiles  
|-- docs/  
|-- configuration-log.md # journal continu  
|-- decisions.md # décisions et justifications  
|-- final-report.pdf # rapport final
```

Les dossiers de données volumineuses (bags, maps, journaux lourds) devront être exclus de Git ou gérés séparément. Les fichiers de configuration, scripts, sources du nœud et documentation doivent au contraire être versionnés.

4. Chemin de réalisation détaillé

Phase 0	Inventaire sans modification Établir l'état réel du PC et du capteur avant toute installation.
Travaux	- Identifier précisément Ubuntu, noyau, CPU, mémoire, stockage, GPU et réseau. - Vérifier Docker, Git, compilateurs et paquets existants. - Brancher le LiDAR seulement selon le manuel et observer les nouveaux périphériques sans changer les règles système. - Créer le journal initial et sauvegarder toutes les sorties utiles.
Validation	Inventaire daté, périphérique ou interface identifié, aucun changement non documenté.
Attention	Ne pas appliquer de règles udev, de chmod global ou de configuration réseau avant d'avoir identifié la liaison réelle.

Phase 1	Conteneur Ubuntu 22.04 + ROS 2 Humble Créer une base isolée et reproductible conforme à Humble.
Travaux	- Installer ou valider Docker sur l'hôte. - Construire une image Jammy avec ROS 2 Humble Desktop ou une sélection minimale documentée. - Monter le workspace, autoriser l'affichage RViz2 et transmettre seulement les périphériques nécessaires. - Tester ros2, colcon et le sourcing de l'environnement.
Validation	Un conteneur reconstruit depuis les fichiers du projet peut lancer ros2, compiler un paquet test et afficher RViz2.
Attention	Éviter les conteneurs privilégiés par défaut. Préférer une liste explicite de périphériques et capacités.

Phase 2	Pilote Unitree L1 Compiler et lancer le pilote officiel ou le port approprié avec une version figée.
Travaux	- Cloner le dépôt dans ros2_ws/src et enregistrer le commit utilisé. - Installer les dépendances avec rosdep ou équivalent documenté. - Compiler avec colcon et inspecter les avertissements. - Configurer le port, l'adresse ou le paramètre réellement détecté, sans supposition.
Validation	Le nœud reste actif sans erreur et publie les messages PointCloud2 et Imu attendus.
Attention	Les noms de topics documentés sont des attentes. Ils doivent être vérifiés sur la version effectivement compilée.

Phase 3	Validation des données 4D Prouver que les données sont cohérentes avant toute cartographie.
Travaux	- Lister nœuds, topics, types et QoS. - Mesurer fréquence, latence approximative et continuité des timestamps. - Inspecter les champs du nuage et la frame_id. - Visualiser le nuage dans RViz2 et vérifier la cohérence de l'IMU lors de mouvements contrôlés.
Validation	Nuage stable et intelligible, IMU réactive, absence de pertes anormales et repères identifiés.
Attention	Un nuage visible ne suffit pas: des timestamps, repères ou unités erronés peuvent rendre le SLAM inutilisable.

Phase 4	Nœud ROS 2 de surveillance Créer un composant simple qui transforme les contrôles manuels en diagnostic reproductible.
Travaux	- S'abonner au nuage et à l'IMU avec un QoS compatible. - Afficher fréquence, âge du dernier message, nombre de points, frames et champs présents. - Signaler données absentes, fréquence trop faible ou timestamp non croissant. - Ajouter tests et paramètres pour les noms de topics.
Validation	Le nœud détecte correctement l'arrêt du pilote et résume les caractéristiques utiles à Point-LIO.
Attention	Ne pas modifier ou filtrer les données avant d'avoir établi une référence brute.

Phase 5	Enregistrement rosbag2 Conserver un jeu de données qui permet de reproduire les essais sans garder le LiDAR branché.
Travaux	- Enregistrer nuage, IMU, tf et tf_static si disponibles. - Noter durée, scène, mouvements et paramètres du pilote. - Rejouer le bag et comparer les fréquences / timestamps. - Conserver un petit bag de validation et un bag de cartographie séparés.
Validation	Le bag se rejoue avec les mêmes types et alimente RViz2 ainsi que le nœud de contrôle.
Attention	Surveiller le débit disque et l'espace libre; les nuages de points peuvent produire rapidement de gros fichiers.

Phase 6	Point-LIO et cartographie Estimer la trajectoire à partir du LiDAR et de l'IMU, puis accumuler la carte.
Travaux	- Choisir et figer la branche ROS 2 compatible L1/Humble. - Adapter uniquement les topics, champs, extrinsèques et paramètres réellement nécessaires. - Tester d'abord sur un bag court, puis sur un parcours de pièce contrôlé. - Observer la dérive, les ruptures de trajectoire et la qualité des surfaces.
Validation	Trajectoire continue, carte géométriquement cohérente et absence de divergence lors du parcours prévu.
Attention	Les extrinsèques LiDAR-IMU, l'échelle temporelle et les champs de points sont des causes majeures d'échec.

Phase 7	Export, contrôle et documentation Produire un livrable réutilisable et explicable à l'équipe.
Travaux	- Exporter la carte PCD et conserver les paramètres exacts. - Contrôler visuellement murs, sols, doublages et zones manquantes. - Écrire la procédure de reconstruction depuis une machine propre. - Finaliser le rapport PDF avec problèmes, solutions et raisons des choix.
Validation	Un autre utilisateur peut reconstruire l'environnement, rejouer les données et retrouver une carte équivalente.
Attention	Ne pas confondre une belle visualisation RViz avec une carte métriquement validée.

5. Contrôles ROS 2 à effectuer

Les commandes ci-dessous sont des exemples prévus pour la phase de validation. Elles ne doivent être exécutées qu'après le lancement de l'environnement Humble et le sourcing correct du workspace.

Commande prévue	Ce qu'elle vérifie	Point d'attention
<code>ros2 node list</code>	Présence des nœuds actifs.	Un nœud absent indique souvent un lancement ou une dépendance en échec.
<code>ros2 topic list -t</code>	Topics et types de messages.	Confirmer les noms réels au lieu de se fier uniquement au README.
<code>ros2 topic info -v /unilidar/cloud</code>	Publishers, subscribers et QoS du nuage.	Un QoS incompatible peut donner l'impression que le topic est vide.
<code>ros2 topic hz /unilidar/cloud</code>	Fréquence effective du nuage.	Mesurer plusieurs dizaines de secondes dans une situation stable.
<code>ros2 topic echo --once /unilidar/imu</code>	Structure et valeurs d'un message IMU.	Éviter un echo continu si le débit est élevé.
<code>ros2 interface show sensor_msgs/msg/PointCloud2</code>	Définition standard du message.	Les champs internes du nuage sont décrits par la liste fields.
<code>ros2 bag record /unilidar/cloud /unilidar/imu /tf /tf_static</code>	Enregistrement des données utiles.	Adapter la liste aux topics existants et surveiller le disque.
<code>colcon build --symlink-install</code>	Compilation du workspace.	Lancer depuis la racine <code>ros2_ws</code> et lire le résumé d'erreurs.

6. Problèmes prévisibles et stratégie de résolution

Problème	Symptôme	Approche de résolution
Humble sur Ubuntu 24.04	Paquets absents ou dépendances incohérentes.	Conteneur Jammy/Humble; ne pas forcer les dépôts Jammy sur l'hôte Noble.
Périphérique inaccessible dans Docker	Le pilote ne voit pas le capteur.	Identifier le périphérique sur l'hôte, vérifier groupe / permissions, puis le transmettre explicitement au conteneur.
Topic visible mais aucune donnée reçue	hz et echo restent silencieux.	Comparer QoS, état du pilote, liaison physique et paramètres.
Nuage incohérent dans RViz2	Points dispersés, fixes ou non affichés.	Vérifier Fixed Frame, frame_id, TF, unités, champs et orientation du capteur.
Point-LIO diverge	Trajectoire cassée ou carte dédoublée.	Contrôler timestamps, synchronisation, extrinsèques, bruit IMU, champs du nuage et paramètres L1.
Bag incomplet	Rejeu différent de l'essai réel.	Inclure tous les topics nécessaires et conserver les paramètres / TF avec les métadonnées.
Débit trop élevé	CPU, mémoire ou disque saturé.	Mesurer avant d'optimiser; réduire visualisation ou enregistrement de manière contrôlée, sans altérer la référence brute.
Remote Control instable	Association expirée ou réponse en timeout.	Continuer localement; réinitialiser uniquement le service Remote et ne pas bloquer la configuration ROS.

7. Conditions générales de travail

Sécurité et autorisations

Aucun mot de passe, token, code d'association ou clé API ne doit être copié dans le journal ou la conversation. Les opérations destructrices, changements système importants et installations doivent être annoncés et approuvés.

Explication des commandes

Avant chaque commande Linux ou ROS: expliquer ce qu'elle fait, pourquoi elle est utilisée, ce qu'elle peut modifier et le résultat attendu. Après exécution: conserver la sortie utile et interpréter les erreurs.

Reproductibilité

Figurer les versions et commits, conserver Dockerfile / compose, paramètres YAML, launch files et commandes colcon. Ne jamais documenter seulement l'état final; documenter aussi le chemin qui y mène.

Protection du PC

Ne pas mélanger les dépôts Ubuntu 22.04 avec l'hôte 24.04. Éviter sudo lorsque ce n'est pas nécessaire. Ne pas appliquer chmod 777 ni lancer le conteneur en mode privilégié sans justification explicite.

Protection du matériel

Brancher et alimenter le LiDAR conformément au manuel Unitree. Stabiliser le câble, éviter les chocs et confirmer la tension / interface. Ne pas déplacer brutalement le capteur pendant les tests initiaux.

Qualité des essais

Commencer capteur immobile, puis effectuer des rotations et translations lentes. Pour la cartographie, fermer des boucles si l'algorithme le permet, éviter les mouvements brusques et noter le parcours.

Gestion des données

Nommer les bags et cartes avec date, scène et configuration. Contrôler l'espace disque. Ne pas versionner automatiquement les gros fichiers ou les données sensibles du laboratoire.

Critère de décision

Une étape n'est terminée que lorsqu'un test reproductible la valide. Une absence d'erreur dans le terminal ou une visualisation ponctuelle ne constitue pas seule une validation.

8. Journal initial des commandes et incidents

Cette section reprend les opérations liées à la préparation de Codex et de Remote Control. Les codes temporaires ont été remplacés par [MASQUÉ].

Commande / événement	Résultat observé	Interprétation
<code>cd ~/unitree_l1_project</code>	Le terminal se place dans le dossier de projet.	Bonne pratique: toutes les actions doivent être rattachées au même dossier.
<code>codex remote-control start</code>	Première tentative: connexion en erreur.	Le service était activé mais l'authentification Codex était absente.
<code>codex remote-control stop</code>	Remote Control stopped.	Arrêt propre du service resté en erreur; aucun fichier ROS modifié.
<code>codex --version</code>	codex-cli 0.144.4	Version identifiée pour le journal et le diagnostic.
<code>coed login status</code>	Commande introuvable.	Simple faute de frappe. La bonne commande est <code>codex</code> .
<code>codex login status</code>	Not logged in.	Cause principale du premier échec mise en évidence.
<code>codex doctor</code>	Installation cohérente; authentification absente; WebSocket en 401; accès HTTPS disponible.	Le 401 était cohérent avec l'absence de jeton. Aucun besoin de réinstaller Codex.
<code>codex login</code>	Successfully logged in.	Authentification avec le compte ChatGPT terminée.
<code>codex remote-control start --json</code>	status: connected; daemon started; version 0.144.4.	Le PC Ubuntu est connecté au relais Remote.
<code>code remote-control pair</code>	Commande code introuvable.	code désigne Visual Studio Code; il ne faut pas l'installer pour cette procédure.
<code>codex remote-control pair</code>	Pairing code: [MASQUÉ]	Code temporaire généré. Il doit être saisi uniquement dans l'application officielle.
<code>codex remote-control pair (nouvel essai)</code>	Timeout waiting for remoteControl/pairing/start.	Une demande pouvait déjà être active; réinitialiser le service et ne lancer qu'une seule association.

8.1 Diagnostic technique conservé

- Système: Ubuntu 24.04 / Noble, 64 bits, locale en_US.UTF-8.
- Codex: installation standalone cohérente; version 0.144.4; Git 2.43.0 disponible.
- État local: bases de données saines; configuration chargée; sandbox et approbations actives.
- Avant login: aucun identifiant Codex trouvé, WebSocket refusé en HTTP 401 et endpoint ChatGPT joignable.
- Après login: Remote Control a répondu connected et a démarré le démon local.
- Point encore ouvert: terminer l'association avec le client ChatGPT sans partager le code.

9. Export structuré de la conversation

La chronologie suivante conserve les décisions et demandes importantes, sans reproduire les secrets ni les répétitions de terminal.

Séquence	Contenu conservé
Définition du projet	Premier petit projet de robotique: configurer le Unitree 4D LiDAR L1 sous ROS 2 Humble.
Réduction du périmètre	Oublier temporairement le Jackal; créer un nœud ROS capable de faire fonctionner le LiDAR, lire ses données 4D et construire une carte.
Attente d'autonomie	Autoriser Codex à travailler sur le PC Ubuntu, tout en expliquant clairement ce qu'il fait et pourquoi.

Séquence	Contenu conservé
Environnement annoncé	Ubuntu 24.04, ROS non encore téléchargé ni installé dans ce nouveau cadre; Codex et ChatGPT connectés.
Exigence documentaire	Créer à la fin un PDF pas à pas sur le LiDAR, l'environnement, les problèmes, les solutions et les raisons des choix.
Problème de modèle	Un modèle portant l'ancien nom gpt-5.3-codex n'était pas accepté avec l'authentification ChatGPT; le diagnostic actuel indique désormais le modèle par défaut.
Besoin de continuité	Éviter de transmettre manuellement toutes les sorties; tentative de connexion Remote entre ChatGPT et Codex sur Ubuntu.
Diagnostic Remote	L'échec initial venait de l'absence d'authentification. Après codex login, le serveur Remote est connecté.
Association	Un code temporaire a été généré puis masqué. Une seconde demande a expiré; la procédure doit être relancée proprement une seule fois.
Demande actuelle	Exporter cette conversation sous forme de PDF facilement imprimable, en expliquant le but, les étapes et les conditions générales.

10. Critères de validation et listes de contrôle

10.1 Avant de brancher ou configurer le LiDAR

- [] Identifier le câble, l'alimentation, l'interface et le manuel correspondant au L1.
- [] Vérifier l'espace disque et sauvegarder les informations système.
- [] Créer le journal docs/configuration-log.md.
- [] Vérifier que le conteneur n'obtient que les droits matériels nécessaires.

10.2 Validation du pilote

- [] Le nœud reste actif pendant au moins un essai continu documenté.
- [] /unilidar/cloud est de type PointCloud2 et contient des points cohérents.
- [] /unilidar/imu est de type Imu et réagit aux mouvements contrôlés.
- [] Les fréquences, timestamps, frames et QoS sont consignés.
- [] RViz2 affiche le nuage avec un repère fixe justifié.

10.3 Validation de la cartographie

- [] Un bag court peut être rejoué sans le LiDAR.
- [] Point-LIO accepte les topics et les champs sans modification opaque.
- [] La trajectoire ne présente pas de rupture ou de dérive évidente.
- [] Les murs et le sol ne sont pas fortement doublés dans la carte.
- [] La carte finale est exportée avec les paramètres et le commit logiciel associés.

10.4 Livrables finaux

- [] Dockerfile et compose.yaml reproductibles.
- [] Workspace ROS 2 organisé et compilable avec colcon.
- [] Nœud I1_monitor documenté et testé.
- [] Bag de validation et carte 3D disponibles selon les règles du laboratoire.
- [] Rapport final PDF mis à jour avec toutes les commandes et décisions.

11. Registre initial des décisions

Décision	Justification	Statut
Travailler d'abord sans le Jackal	Réduire les variables et valider le capteur indépendamment.	Confirmée
Conserver ROS 2 Humble	Exigence du projet et compatibilité attendue avec les ressources Unitree / Point-LIO choisies.	Confirmée
Privilégier Docker Jammy sur hôte Noble	Humble cible Ubuntu 22.04; éviter de contaminer Ubuntu 24.04 avec des dépôts incompatibles.	À valider après inventaire
Utiliser d'abord le pilote Unitree	Partir du chemin supporté par le fabricant avant d'écrire une couche personnalisée.	Planifiée
Créer un nœud de surveillance séparé	Contrôler la qualité sans modifier le pilote ni les données brutes.	Planifiée
Enregistrer avant de régler Point-LIO	Rendre le diagnostic reproductible et éviter de dépendre du capteur en permanence.	Planifiée
Documenter en continu	Le rapport final doit expliquer les problèmes et les raisons, pas seulement la recette réussie.	Confirmée
Remote ne doit pas bloquer ROS	La connexion distante est un confort; le projet reste exécutable localement.	Confirmée

12. Références techniques de départ

Ces liens constituent le point de départ documentaire. Les versions et commits réellement utilisés devront être ajoutés au journal lors de l'installation.

- ROS 2 Humble - installation Ubuntu 22.04:
<https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>
- ROS 2 Humble - tutoriels: <https://docs.ros.org/en/humble/Tutorials.html>
- Unitree UniLiDAR SDK: https://github.com/unitreerobotics/unilidar_sdk
- README ROS 2 du pilote Unitree:
https://github.com/unitreerobotics/unilidar_sdk/tree/main/unitree_lidar_ros2
- Manuel du Unitree 4D LiDAR L1:
<https://oss-global-cdn.unitree.com/static/52b72f707b304d229d4321eea223738f.pdf>
- Point-LIO ROS 2, port communautaire: https://github.com/dfloreaa/point_lio_ros2
- Point-LIO Unitree officiel, référence ROS 1: https://github.com/unitreerobotics/point_lio_unilidar
- Codex CLI: <https://developers.openai.com/codex/cli>
- ChatGPT Remote connections: <https://learn.chatgpt.com/docs/remote-connections>

Prochaine action recommandée. Finaliser ou abandonner provisoirement l'association Remote, puis lancer dans `~/unitree_l1_project` une inspection strictement en lecture seule. Le premier résultat technique attendu n'est pas l'installation de ROS: c'est un inventaire daté de l'hôte et de l'interface réelle du LiDAR, suivi d'une décision confirmée sur Docker.